FP/OO Multiparadigm Design Technique — Quick Reference

Universidad del Cauca · Undergraduate Thesis · 2026

1. OO / FUNCTIONAL DECISION TABLE

Problem characteristic	Approach	Justification
Entity with identity and mutable state	OO (encapsulation)	Protects state integrity
Data transformation pipeline (ETL)	Functional (pipelines)	Modularity via composition
Simple parameterizable behavior	Functional (lambdas)	Lightweight configurability
Complex parameterizable behavior	OO (interfaces / Strategy)	Polymorphism with state
Frequent new data types	OO (open hierarchies)	Extensibility via subtype
Frequent new operations	Functional (ADT + pattern matching)	Extensibility via functions
New types and operations simultaneously	Hybrid (object algebra)	Solves the Expression Problem
High concurrency	Functional (immutability)	Eliminates side effects

2. ABSTRACTION MECHANISMS M1–M5

Mech.	Name	Key question
M1	Object Algebras / Tagless Final	How do I grow types AND operations without touching existing code?
M2	Type Classes / Generalized Interfaces	How do I add behavior to existing types without modifying them?
M3	Sealed ADT + Pattern Matching	How do I decompose by case without Visitor scaffolding?
M4	HOF + Closures + FP Composition	How do I represent pure behavior without class hierarchies?
M5	Signals / Actors / Continuations	How do I propagate changes without coupling subjects to observers?

3. THE 6 STEPS OF THE TECHNIQUE

#	Step	Description	Output
1	**Decomposition & classification**	Break requirements into components and apply Table 1 to each.	OO vs. FP list
2	**Existing pattern detection**	Inventory patterns (Strategy, Visitor, Observer…) and map to M1–M5.	Pattern → M1–M5 map
3	**Hybrid interface definition**	Design OO interfaces; at variation points, replace with function params.	OO contracts with FP variation
4	**Visual modeling**	Build diagrams with the lightweight UML profile (Table 4).	Diagrams with explicit paradigm
5	**Refactoring toward immutability**	Reduce mutable state; convert «pure» classes into ValueObjects.	Minimal mutable state
6	**O(1) vs. O(N) validation**	Verify anticipated changes resolve with bounded cost.	Validated design

4. LIGHTWEIGHT UML PROFILE STEREOTYPES

Stereotype	Meaning
«pure»	Side-effect-free method; referentially transparent
«ValueObject»	Immutable class representing a value (not an entity)
«signal»	Reactive / observable attribute
«var»	Mutable attribute with explicit state
{immutable}	Fields do not change after construction